

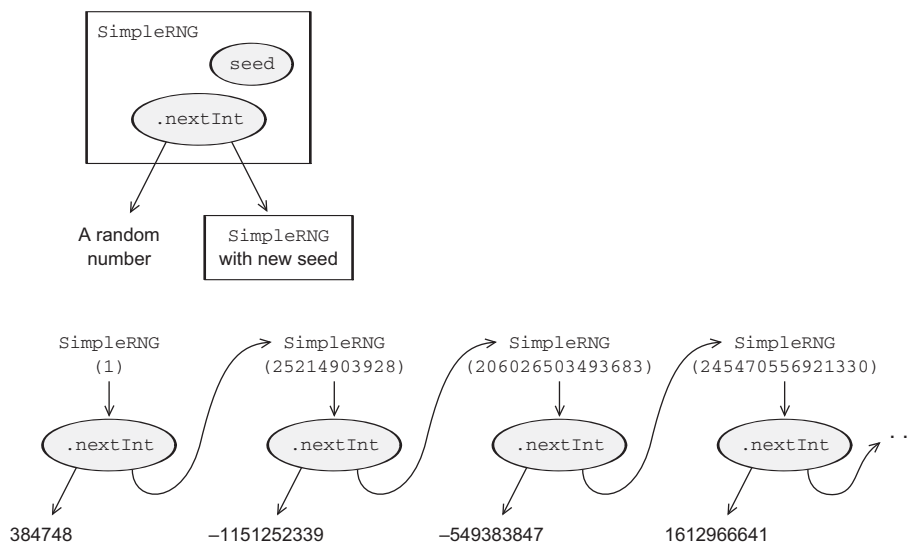
6.2 Purely functional random number generation

The key to recovering referential transparency is to make the state updates *explicit*. Don't update the state as a side effect, but simply return the new state along with the value that we're generating. Here's one possible interface to a random number generator:

```
trait RNG {
  def nextInt: (Int, RNG)
}
```

This method should generate a random `Int`. We'll later define other functions in terms of `nextInt`. Rather than returning only the generated random number (as is done in `scala.util.Random`) and updating some internal state by *mutating* it in place, we return the random number and the new state, leaving the old state unmodified.³ In effect, we separate the concern of *computing* what the next state is from the concern of *communicating* the new state to the rest of the program. No global mutable memory is being used—we simply return the next state back to the caller. This leaves the caller of `nextInt` in complete control of what to do with the new state. Note that we're still *encapsulating* the state, in the sense that users of this API don't know anything about the implementation of the random number generator itself.

A functional RNG



Each call to `SimpleRNG.nextInt` returns the next random number in the sequence and the `SimpleRNG` object needed to continue the sequence.

³ Recall that (A, B) is the type of two-element tuples, and given $p: (A, B)$, you can use $p._1$ to extract the A and $p._2$ to extract the B .